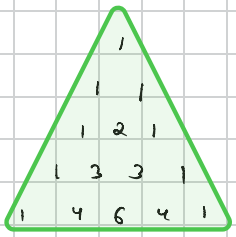
118. Pascal's Triangle (Easy) (Arrays)

Given an integer numRows, return the first numRows of Pascal's triangle. In Pascal's triangle, each number is the sum of the two numbers directly above it as shown:



Example 1:
Input: numRows = 5
Output: [[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]
Example 2:
Input: numRows = 1
Output: [[1]]

Code:-

```
public long nCr(int n, int r) {
    if (n < 0 || r < 0 || r > n) {
        throw new IllegalArgumentException("Require n >= 0 and 0 <= r <= n");
    }

    r = Math.min(r, n - r);
    long result = 1;

    for (int i = 0; i < r; i++) {
        result = result * (n - i) / (i + 1);
    }

    return result;
}

public List<List<Integer>> generate(int numRows) {
    List<List<Integer>> triangle = new ArrayList<>();

    for (int row = 0; row < numRows; row++) {
        List<Integer> currentRow = new ArrayList<>();

        for (int col = 0; col <= row; col++) {
            currentRow.add((int) nCr(row, col));
        }

        triangle.add(currentRow);
    }

    return triangle;
}
```

→ Just maths ^nC_r $^nC_r = \frac{n!}{(n-r)!\,r!}$

numRows = 5

① row = 0 col = 0
currRow = 1 add = {1}
triangle = {{1}} add
triangle: 1

② row = 1 col = 0
currRow = 1 = {1} add
col = 1
currRow ⇒ 1 ⇒ {1,1} add
triangle = {{1},{1,1}} add
triangle:
1
1 1

③ row = 2 col = 0
currRow = {1} add
col = 1
currRow = $^2C_1$ ⇒ {1,2} add
col = 2
currRow ⇒ {1,2,1} add
triangle = {{1},{1,1},{1,2,1}} add
triangle:
1
1 1
1 2 1

④ row = 3 col = 0 currRow = {1} add
col = 1 currRow ⇒ {1, $^3C_1$} ⇒ {1,3} add
col = 2 currRow ⇒ {1,3, $^3C_2$} ⇒ {1,3,3} add
col = 3 currRow ⇒ {1,3,3,1} add
triangle = {{1},{1,1},{1,2,1},{1,3,3,1}} add

1
1 1
1 2 1
1 3 3 1

⑤ row = 4 col = 0 currRow = {1} add
col = 1 currRow ⇒ {1, $^4C_1$} ⇒ {1,4} add
col = 2 currRow ⇒ {1,4, $^4C_2$} ⇒ {1,4,6} add
col = 3 currRow ⇒ {1,4,6, $^4C_3$} ⇒ {1,4,6,4} add
col = 4 currRow ⇒ {1,4,6,4,1} add
triangle = {{1},{1,1},{1,2,1},{1,3,3,1},{1,4,6,4,1}} add

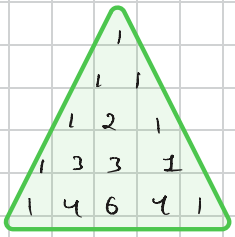
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1

27

14. Pascal's Triangle – II (Easy) (Arrays)

Given an integer rowIndex, return the rowIndexth (0-indexed) row of the Pascal's triangle. In Pascal's triangle, each number is the sum of the two numbers directly above it as shown:

Example 1:
Input: rowIndex = 3
Output: [1,3,3,1]
Example 2:
Input: rowIndex = 0
Output: [1]
Example 3:
Input: rowIndex = 1
Output: [1,1]



Code :–

```
public long nCr(int n, int r) {
    if (n < 0 || r < 0 || r > n) {
        throw new IllegalArgumentException("Require n >= 0 and 0 <= r <= n");
    }

    r = Math.min(r, n - r);
    long result = 1;

    for (int i = 0; i < r; i++) {
        result = result * (n - i) / (i + 1);
    }

    return result;
}

public List<Integer> getRow(int rowIndex) {

    List<Integer> row = new ArrayList<>();

    for (int col = 0; col <= rowIndex; col++) {
        row.add((int) nCr(rowIndex, col));

    }

    return row;

}
```

1) rowIndex = 3
col = 0 row = {1,
col = 1 row = {1, 3C1,
col = 2 row = {1, 3, 3C2,
col = 3 row = {1, 3, 3, 1}

2) rowIndex = 0
row = {1}

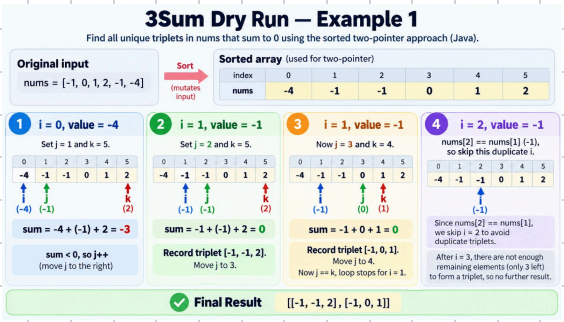
3) rowIndex = 1
row = {1,
row = {1, 1}

28

15. 3 Sum (Medium) (Arrays, Two Pointers)

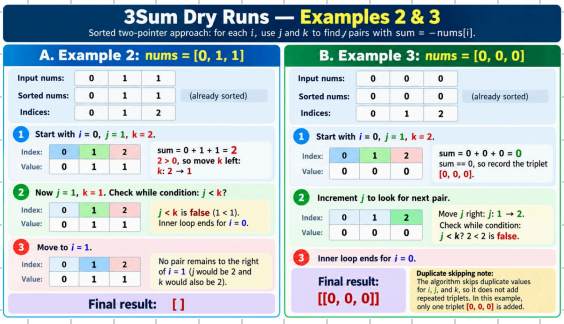
Given an integer array nums, return all the triplets [nums[i], nums[j], nums[k]] such that i != j, i != k, and j != k, and nums[i] + nums[j] + nums[k] == 0. Notice that the solution set must not contain duplicate triplets.

Example 1:
Input: nums = [-1,0,1,2,-1,-4]
Output: [[-1,-1,2],[-1,0,1]]
Explanation:
nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.
nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.
nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.
The distinct triplets are [-1,0,1] and [-1,-1,2].
Notice that the order of the output and the order of the triplets does not matter.
Example 2:
Input: nums = [0,1,1]
Output: []
Explanation: The only possible triplet does not sum up to 0.
Example 3:
Input: nums = [0,0,0]
Output: [[0,0,0]]
Explanation: The only possible triplet sums up to 0.



Code:–

```
public List<List<Integer>> threeSum(int[] nums) {
    List<List<Integer>> res = new ArrayList<>();
    Arrays.sort(nums);

    for (int i = 0; i < nums.length; i++) {
        if (i > 0 && nums[i] == nums[i-1]) {
            continue;
        }

        int j = i + 1;
        int k = nums.length - 1;

        while (j < k) {
            int total = nums[i] + nums[j] + nums[k];

            if (total > 0) {
                k--;
            } else if (total < 0) {
                j++;
            } else {
                res.add(Arrays.asList(nums[i], nums[j], nums[k]));
                j++;

                while (nums[j] == nums[j-1] && j < k) {
                    j++;
                }
            }
        }
    }
    return res;
}
```



29

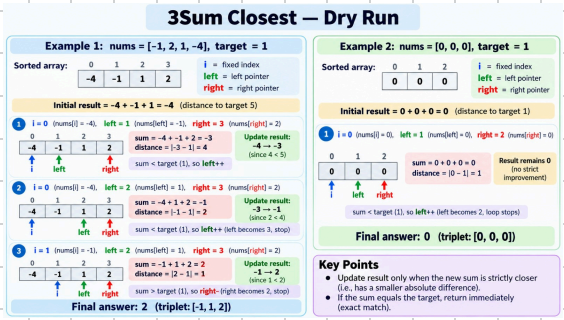
16. 3 Sum Closest (Medium) (Arrays, Two Pointers)

You are given an integer array nums of length n and an integer target. Find three integers at distinct indices in nums such that the sum is closest to target.
Return the sum of the three integers. You may assume that each input would have exactly one solution.

Example 1:
Input: nums = [-1,2,1,-4], target = 1
Output: 2
Explanation: The sum that is closest to the target is 2. (-1 + 2 + 1 = 2).
Example 2:
Input: nums = [0,0,0], target = 1
Output: 0
Explanation: The sum that is closest to the target is 0. (0 + 0 + 0 = 0).

Code :-

```
public int threeSumClosest(int[] nums, int target) {
        Arrays.sort(nums);
        int n = nums.length;
        int result = nums[0] + nums[1] + nums[2]; // Initial best guess

        for (int i = 0; i < n - 2; i++) {
            int left = i + 1, right = n - 1;

            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];

                if (Math.abs(target - sum) < Math.abs(target - result)) {
                    result = sum;
                }

                if (sum == target) return target;
                else if (sum < target) left++;
                else right--;
            }
        }

        return result;
    }
```

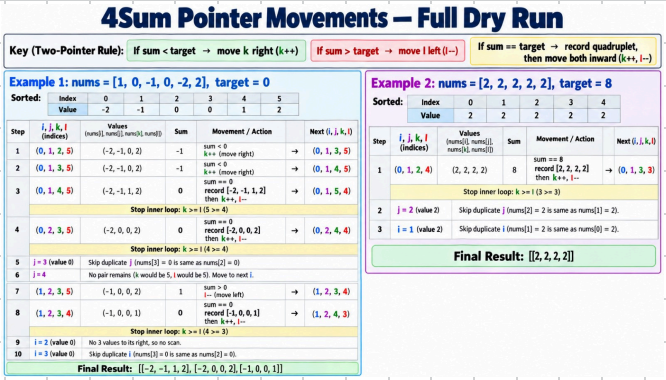


90

18. 4 Sum (Medium) (Arrays, Two Pointers)

Given an array nums of n integers, return an array of all the unique quadruplets [nums[a], nums[b], nums[c], nums[d]] such that:
0 <= a, b, c, d < n
a, b, c, and d are distinct.
nums[a] + nums[b] + nums[c] + nums[d] == target
You may return the answer in any order.

Example 1:
Input: nums = [1,0,-1,0,-2,2], target = 0
Output: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]
Example 2:
Input: nums = [2,2,2,2,2], target = 8
Output: [[2,2,2,2]]



Code :-

```
public List<List<Integer>> fourSum(int[] nums, int target) {
        List<List<Integer>> ans = new ArrayList<>();
        int n = nums.length;
        Arrays.sort(nums);
        for (int i = 0; i < n; i++) {
            if (i > 0 && nums[i] == nums[i-1]) continue;
            for (int j = i + 1; j < n; j++) {
                if (j > i + 1 && nums[j] == nums[j-1]) continue;
                int k = j + 1;
                int l = n - 1;
                while (k < l) {
                    long sum = (long) nums[i] + nums[j] + nums[k] + nums[l];
                    if (sum < target) k++;
                    else if (sum > target) l--;
                    else { // sum == target
                        List<Integer> temp = Arrays.asList(nums[i], nums[j], nums[k], nums[l]);
                        ans.add(temp);
                        k++;
                        l--;
                        while (k < l && nums[k] == nums[k-1]) k++;
                        while (k < l && nums[l] == nums[l+1]) l--;
                    }
                }
            }
        }
        return ans;
    }
```

91

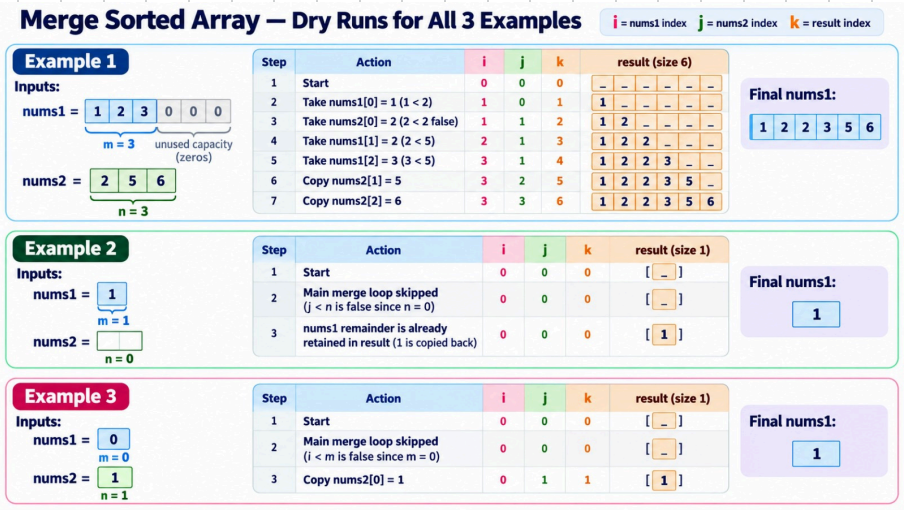
88. Merge Sorted Array (Easy) (Arrays, Two Pointers)

You are given two integer arrays nums1 and nums2, sorted in non-decreasing order, and two integers m and n, representing the number of elements in nums1 and nums2 respectively. Merge nums1 and nums2 into a single array sorted in non-decreasing order. The final sorted array should not be returned by the function, but instead be stored inside the array nums1. To accommodate this, nums1 has a length of m + n, where the first m elements denote the elements that should be merged, and the last n elements are set to 0 and should be ignored. nums2 has a length of n.

Example 1:
Input: nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3
Output: [1,2,2,3,5,6]
Explanation: The arrays we are merging are [1,2,3] and [2,5,6].
The result of the merge is [1,2,2,3,5,6] with the underlined elements coming from nums1.
Example 2:
Input: nums1 = [1], m = 1, nums2 = [], n = 0
Output: [1]
Explanation: The arrays we are merging are [1] and [].
The result of the merge is [1].
Example 3:
Input: nums1 = [0], m = 0, nums2 = [1], n = 1
Output: [1]
Explanation: The arrays we are merging are [] and [1].
The result of the merge is [1].
Note that because m = 0, there are no elements in nums1. The 0 is only there to ensure the merge result can fit in nums1.

Code :-

```
public void merge(int[] nums1, int m, int[] nums2, int n) {

    int[] result = new int[n + m];

    int i = 0;
    int j = 0;
    int k = 0;

    while( i < m && j < n){
        if(nums1[i] < nums2[j]){
            result[k] = nums1[i];
            i++;
        }else{
            result[k] = nums2[j];
            j++;
        }
        k++;
    }

    while( i < m ){
        result[k] = nums1[i];
        i++;
        k++;
    }

    while( j < n){
        result[k] = nums2[j];
        j++;
        k++;
    }

    for(int x = 0 ; x < m + n; x++){
        nums1[x] = result[x];
    }

}
```



32

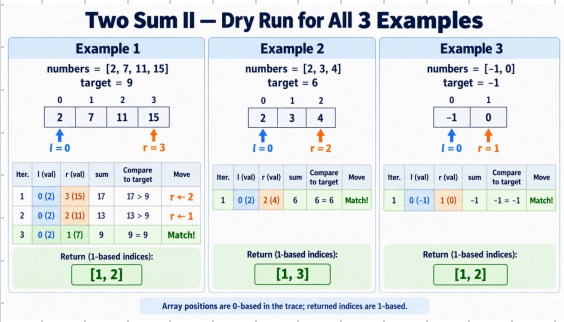
167. Two Sum – II – Input Array is Sorted (Medium) (Arrays, Two Pointers)

You are given a 1-indexed array of integers numbers that is already sorted in non-decreasing order. Find two numbers such that they add up to a specific target number. Let these two numbers be numbers[index1] and numbers[index2] where 1 <= index1 < index2 <= numbers.length.Return the indices of the two numbers index1 and index2 as an integer array [index1, index2] of length 2. The tests are generated such that there is exactly one solution. You may not use the same element twice.
Your solution must use only constant extra space.

Example 1:
Input: numbers = [2,7,11,15], target = 9
Output: [1,2]
Explanation: The sum of 2 and 7 is 9. Therefore, index1 = 1, index2 = 2. We return [1, 2].
Example 2:
Input: numbers = [2,3,4], target = 6
Output: [1,3]
Explanation: The sum of 2 and 4 is 6. Therefore index1 = 1, index2 = 3. We return [1, 3].
Example 3:
Input: numbers = [-1,0], target = -1
Output: [1,2]
Explanation: The sum of -1 and 0 is -1. Therefore index1 = 1, index2 = 2. We return [1, 2].

Code :-

```
public int[] twoSum(int[] numbers, int target) {

    int[] result = new int[2];

    int n = numbers.length;
    int l = 0;
    int r = n -1;

    while( l < r){
        int sum = numbers[l] + numbers[r];

        if(sum == target){
            result[0] = l + 1;
            result[1] = r + 1;
            return result;
        }

        if(sum < target)l++;
        if(sum > target)r--;
    }

    return result;
}
```



33

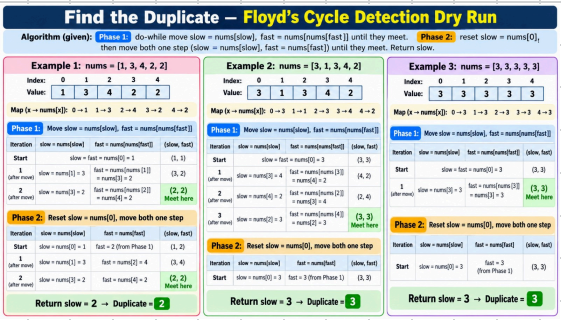
287. Find the Duplicate Number (Medium) (Arrays, Two Pointers)

Given an array of integers nums containing n + 1 integers where each integer is in the range [1, n] inclusive. There is only one repeated number in nums, return this repeated number. You must solve the problem without modifying the array nums and using only constant extra space.

Example 1:
Input: nums = [1,3,4,2,2]
Output: 2
Example 2:
Input: nums = [3,1,3,4,2]
Output: 3
Example 3:
Input: nums = [3,3,3,3,3]
Output: 3

Code :-

```
public int findDuplicate(int[] nums) {
    int slow = nums[0];
    int fast = nums[0];

    do{
        slow = nums[slow];
        fast = nums[nums[fast]];
    }
    while(slow != fast);

    slow = nums[0];

    while(slow != fast){
        slow = nums[slow];
        fast = nums[fast];
    }

    return slow;
}
```



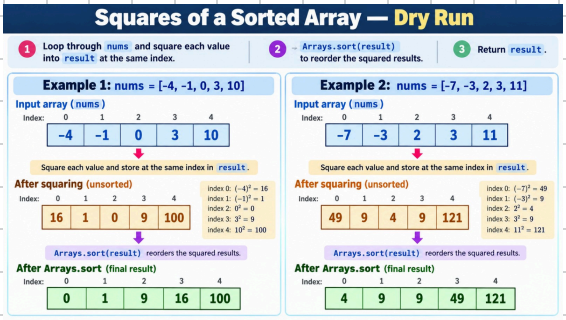
34. 977. Squares of a Sorted Array (Easy) (Arrays)

Given an integer array nums sorted in non-decreasing order, return an array of the squares of each number sorted in non-decreasing order.

Example 1:
Input: nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: After squaring, the array becomes [16,1,0,9,100].
After sorting, it becomes [0,1,9,16,100].
Example 2:
Input: nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]

Code:-

```
public int[] sortedSquares(int[] nums) {
    int[] result = new int[nums.length];

    for(int i = 0; i < nums.length; i++){
        result[i] = nums[i]*nums[i];
    }

    Arrays.sort(result);

    return result;
}
```



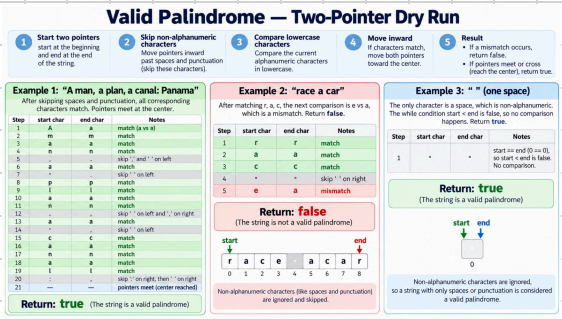
35. 125. Valid Palindrome (Easy) (Strings and Two Pointers)

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers. Given a string s, return true if it is a palindrome, or false otherwise.

Example 1:
Input: s = "A man, a plan, a canal: Panama"
Output: true
Explanation: "amanaplanacanalpanama" is a palindrome.
Example 2:
Input: s = "race a car"
Output: false
Explanation: "raceacar" is not a palindrome.
Example 3:
Input: s = " "
Output: true
Explanation: s is an empty string "" after removing non-alphanumeric characters.
Since an empty string reads the same forward and backward, it is a palindrome.

Code:-

```
public boolean isPalindrome(String s) {
    int start = 0;
    int end = s.length() - 1;

    if(s.length() == 0 || s.length() == 1) return true;

    while (start < end) {

        while (start < end && !Character.isLetterOrDigit(s.charAt(start))) {
            start++;
        }

        while (start < end && !Character.isLetterOrDigit(s.charAt(end))) {
            end--;
        }

        if (Character.toLowerCase(s.charAt(start)) !=
            Character.toLowerCase(s.charAt(end))) {
            return false;
        }

        start++;
        end--;
    }

    return true;
}
```



36. 344. Reverse String (Easy) (Strings, Swapping Algorithm) ← No Need Hand Drawn Visualization / AI Generates Dry run Image

Write a function that reverses a string. The input string is given as an array of characters s. You must do this by modifying the input array in-place with O(1) extra memory.

Example 1:
Input: s = ["h","e","l","l","o"]
Output: ["o","l","l","e","h"]
Example 2:
Input: s = ["H","a","n","n","a","h"]
Output: ["h","a","n","n","a","H"]

Code:-

```
public void reverseString(char[] s) {
    int start = 0;
    int end = s.length - 1;

    while(start < end){
        char temp = s[start];
        s[start] = s[end];
        s[end] = temp;

        start++;
        end--;
    }

}
```

37. 443. String Compression (Medium) (Strings)

Given an array of characters chars, compress it using the following algorithm:
Begin with an empty string s. For each group of consecutive repeating characters in chars:
If the group's length is 1, append the character to s. Otherwise, append the character followed by the group's length. The compressed string s should not be returned separately, but instead, be stored in the input character array chars. Note that group lengths that are 10 or longer will be split into multiple characters in chars. After you are done modifying the input array, return the new length of the array. You must write an algorithm that uses only constant extra space.
Note: The characters in the array beyond the returned length do not matter and should be ignored.

Example 1:
Input: chars = ["a","a","b","b","c","c","c"]
Output: 6
Explanation: The groups are "aa", "bb", and "ccc". This compresses to "a2b2c3".
After modifying the input array in-place, the first 6 characters of chars should be ["a","2","b","2","c","3"].
Example 2:
Input: chars = ["a"]
Output: 1
Explanation: The only group is "a", which remains uncompressed since it is a single character.
After modifying the input array in-place, the first character of chars should be ["a"].
Example 3:
Input: chars = ["a","b","b","b","b","b","b","b","b","b","b","b","b"]
Output: 4
Explanation: The groups are "a" and "bbbbbbbbbbbb". This compresses to "ab12".
After modifying the input array in-place, the first 4 characters of chars should be ["a","b","1","2"].

Code:-

```
public static int compress(char[] chars) {
    int ans = 0;

    for (int i = 0; i < chars.length;) {
        char letter = chars[i];
        int count = 0;

        while (i < chars.length && chars[i] == letter) {
            count++;
            i++;
        }

        chars[ans++] = letter;

        if (count > 1) {
            for (char c : String.valueOf(count).toCharArray()) {
                chars[ans++] = c;
            }
        }
    }

    return ans;
}
```

In-place String Compression — Dry Runs

Example 1 Input: [a , a , b , b , c , c , c] Output: 6

Step	Action	Array state (prefix up to ans)	i	ans
1	Start	a a b b c c c	0	0
2	Process run aa: count = 2 Write a, 2	a 2	2	2
3	Process run bb: count = 2 Write b, 2	a 2 b 2	4	4
4	Process run ccc: count = 3 Write c, 3	a 2 b 2 c 3	7	6

Example 2 Input: [a] Output: 1

Step	Action	Array state (prefix up to ans)	i	ans
1	Start	a	0	0
2	Process a: count = 1 Write a only	a	1	1

Example 3 Input: [a , b , b , b , b , b , b , b , b , b , b , b , b] Output: 4

Step	Action	Array state (prefix up to ans)	i	ans
1	Start	a b b b b b b b b b b b b	0	0
2	Process a: count = 1 Write a only	a	1	1
3	Process bbbbbbbbbbbb: count = 12 Write b, 1, 2	a b 1 2	13	4

i = next unread position (read pointer) ans = next write position (write pointer)

38. Reverse Words in a String - III (Easy) (Strings)

Given a string s, reverse the order of characters in each word within a sentence while still preserving whitespace and initial word order.

Example 1:
Input: s = "Let's take LeetCode contest"
Output: "s'teL ekat edoCteeL tsetnoc"
Example 2:
Input: s = "Mr Ding"
Output: "rM gniD"

Code:-

```
public String reverseWords(String s) {
    String[] words = s.split(" ");
    StringBuilder result = new StringBuilder();
    for (String word : words) {
        StringBuilder reversedWord = new StringBuilder(word).reverse();
        result.append(reversedWord).append(" ");
    }
    result.deleteCharAt(result.length() - 1);

    return result.toString();
}
```

Reverse Words in a String — Dry Run

Key Idea: each word is reversed independently; word order stays the same.

Example 1 Input: "Let's take LeetCode contest"

1. Split on spaces → words = ["Let's" , "take" , "LeetCode" , "contest"]

Step	Current word	Reversed word	Result builder (after appending word + space)
2. Reverse 1st word	"Let's"	"s'teL"	"s'teL "
3. Reverse 2nd word	"take"	"ekat"	"s'teL ekat "
4. Reverse 3rd word	"LeetCode"	"edoCteeL"	"s'teL ekat edoCteeL "
5. Reverse 4th word	"contest"	"tsetnoc"	"s'teL ekat edoCteeL tsetnoc "

6. Delete final trailing space → Output: "s'teL ekat edoCteeL tsetnoc"

Example 2 Input: "Mr Ding"

1. Split on spaces → words = ["Mr" , "Ding"]

Step	Current word	Reversed word	Result builder (after appending word + space)
2. Reverse 1st word	"Mr"	"rM"	"rM "
3. Reverse 2nd word	"Ding"	"gniD"	"rM gniD "

4. Delete final trailing space → Output: "rM gniD"

39. Valid Palindrome II (Easy) (Strings, Two Pointers)

Given a string s, return true if the s can be palindrome after deleting at most one character from it.

Example 1:
Input: s = "aba"
Output: true
Example 2:
Input: s = "abca"
Output: true
Explanation: You could delete the character 'c'.
Example 3:
Input: s = "abc"
Output: false

Valid Palindrome II — Dry Run

```
public boolean validPalindrome(String s) {
    int start = 0, end = s.length() - 1;
    while (start < end) {
        if (s.charAt(start) == s.charAt(end)) {
            start++; end--;
        } else {
            return isPalindrome(s, start + 1, end) || isPalindrome(s, start, end - 1);
        }
    }
    return true;
}

private boolean isPalindrome(String s, int i, int j) {
    while (i < j) {
        if (s.charAt(i) != s.charAt(j)) return false;
        i++; j--;
    }
    return true;
}
```

Example 1: s = "aba" Output: true (No deletion needed)

1. Initial state: start = 0, end = 2 — s[0] = 'a', s[2] = 'a' — Match! Move inward.
2. After moving: start = 1, end = 1 — start >= end (1 >= 1) Loop stops.

Return: true — The string is already a palindrome. No deletion needed.

Example 2: s = "abca" Output: true (Delete one character)

1. Initial state: start = 0, end = 3 — s[0] = 'a', s[3] = 'a' — Match! Move inward.
2. After moving: start = 1, end = 2 — s[1] = 'b', s[2] = 'c' — Mismatch! Try skipping left or right.

Branch A: Skip left (delete index 1) — Check isPalindrome(s, 2, 2) — start = 2, end = 2 — Substring [2..2] = "c" (single character) → true

Branch B: Skip right (delete index 2) — Check isPalindrome(s, 1, 1) — start = 1, end = 1 — Substring [1..1] = "b" (single character) → true

Result: true || true = true
Explanation: Deleting 'c' (index 2) gives "aba". (Also, deleting 'b' gives "aca".)

Example 3: s = "abc" Output: false (Cannot be a palindrome)

1. Initial state: start = 0, end = 2 — s[0] = 'a', s[2] = 'c' — Mismatch! Try skipping left or right.

Branch A: Skip left (delete index 0) — Check isPalindrome(s, 1, 2) — Substring [1..2] = "bc" — Compare s[0] = 'b' with s[2] = 'c', 'b' != 'c' → false — Branch A returns false

Branch B: Skip right (delete index 2) — Check isPalindrome(s, 0, 1) — Substring [0..1] = "ab" — Compare s[0] = 'a' with s[1] = 'b', 'a' != 'b' → false — Branch B returns false

Result: false || false = false
Explanation: Deleting 'a' gives "bc" (not a palindrome), and deleting 'c' gives "ab" (not a palindrome).

Code:-

```
private static boolean isPalindrome(String s,
int start, int end) {

    while (start < end) {

        if (s.charAt(start) != s.charAt(end)) {
            return false;
        }

        start++;
        end--;
    }

    return true;
}

  public static boolean
validPalindrome(String s) {

    int start = 0;
    int end = s.length() - 1;

    while (start < end) {

        if (s.charAt(start) != s.charAt(end)) {

            return isPalindrome(s, start + 1,
end) ||
                isPalindrome(s, start, end - 1);
        }

        start++;
        end--;
    }

    return true;
}
```

(40)

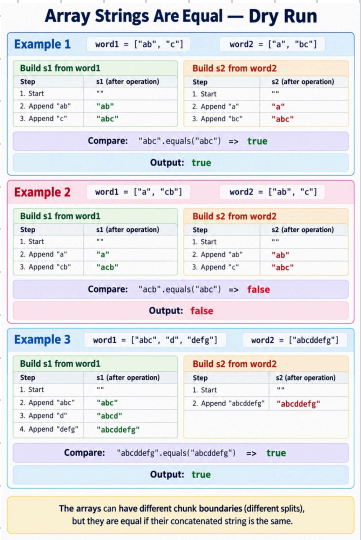
1662. Check If Two String Arrays are Equivalent (Easy) (Strings)

Given two string arrays word1 and word2, return true if the two arrays represent the same string, and false otherwise. A string is represented by an array if the array elements concatenated in order forms the string.

Code:-

```
public boolean arrayStringsAreEqual(String[]
word1, String[] word2) {
    String s1 = "";
    String s2 = "";

    for(int i = 0 ; i < word1.length; i++){
        s1 += word1[i];
    }

    for(int i = 0; i < word2.length; i++){
        s2 += word2[i];
    }

    return s1.equals(s2);
}
```

Example 1:
Input: word1 = ["ab", "c"], word2 = ["a", "bc"]
Output: true
Explanation:
word1 represents string "ab" + "c" -> "abc"
word2 represents string "a" + "bc" -> "abc"
The strings are the same, so return true.
Example 2:
Input: word1 = ["a", "cb"], word2 = ["ab", "c"]
Output: false
Example 3:
Input: word1 = ["abc", "d", "defg"], word2 = ["abcddefg"]
Output: true



(41)

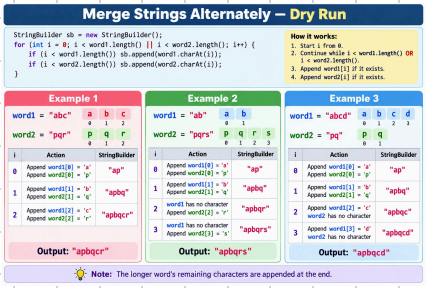
1768. Merge Strings Alternatively (Easy) (Strings)

You are given two strings word1 and word2. Merge the strings by adding letters in alternating order, starting with word1. If a string is longer than the other, append the additional letters onto the end of the merged string. Return the merged string.

Example 1:
Input: word1 = "abc", word2 = "pqr"
Output: "apbqcr"
Explanation: The merged string will be merged as so:
word1: a b c
word2: p q r
merged: a p b q c r
Example 2:
Input: word1 = "ab", word2 = "pqrs"
Output: "apbqrs"
Explanation: Notice that as word2 is longer, "rs" is appended to the end.
word1: a b
word2: p q r s
merged: a p b q r s
Example 3:
Input: word1 = "abcd", word2 = "pq"
Output: "apbqcd"
Explanation: Notice that as word1 is longer, "cd" is appended to the end.
word1: a b c d
word2: p q
merged: a p b q c d

Code:-

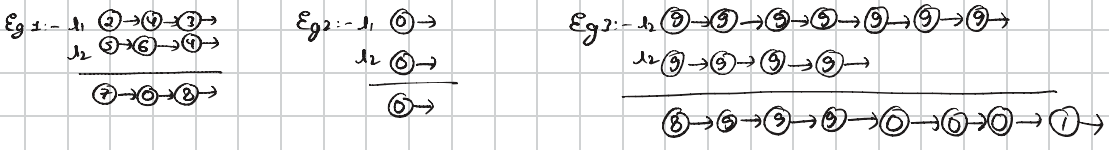
```
public String mergeAlternately(String word1, String word2) {
    StringBuilder sb = new StringBuilder();

    for(int i = 0; i < word1.length() || i < word2.length() ; i++){
        if(i < word1.length()){
            sb.append(word1.charAt(i));
        }

        if(i < word2.length()){
            sb.append(word2.charAt(i));
        }
    }

    return sb.toString();

}
```

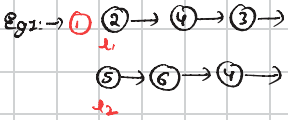


(42)

2. Add Two Numbers (Medium) (Linked List, Dummy Node)

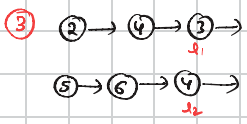
You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list. You may assume the two numbers do not contain any leading zero, except the number 0 itself.





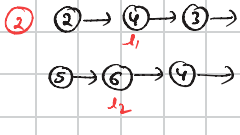
Sum = p + q + carry = 2 + 5 + 0 = 7
Carry = 7/10 = 0

7 →



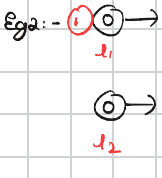
Sum = p + q + carry = 3 + 4 + 1 = 8
Carry = 8/10 = 0

7 → 0 → 8 →



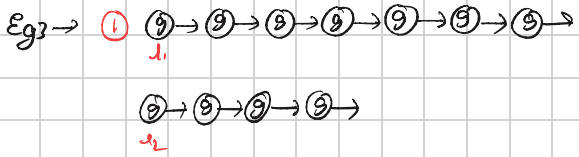
Sum = p + q + carry = 4 + 6 + 0 = 10
Carry = 10/10 = 1

7 → 0 →

Eg2:- 

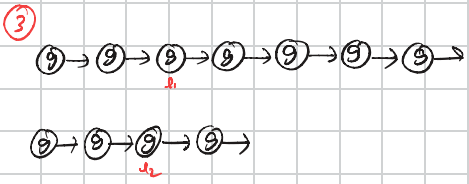
Sum = 0+0+0 = 0

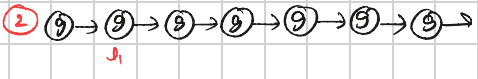
carry = 0/10 = 0

Eg3 → 

Sum = 9+9 = 18

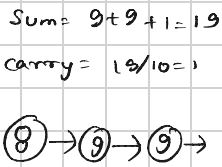
carry = 18/10 = 1





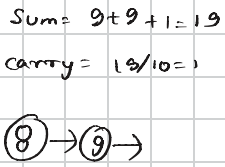
Sum = 9+9+1 = 19

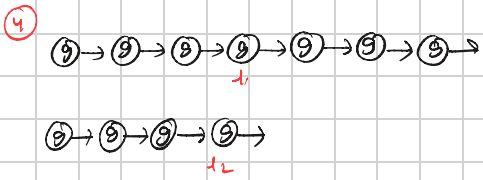
carry = 19/10 = 1

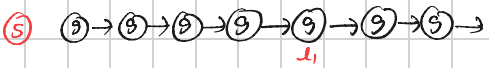


Sum = 9+9+1 = 19

carry = 19/10 = 1

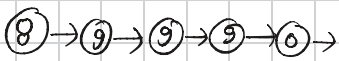


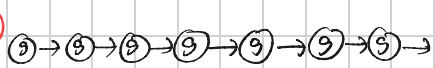




Sum = 9+0+1 = 10

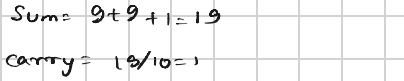
carry = 10/10 = 1

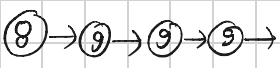




Sum = 9+9+1 = 19

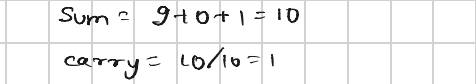
carry = 19/10 = 1

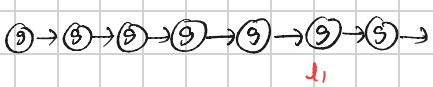




Sum = 9+0+1 = 10

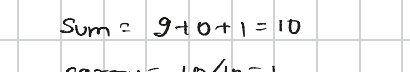
carry = 10/10 = 1

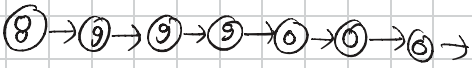




Sum = 9+0+1 = 10

carry = 10/10 = 1





Code:-

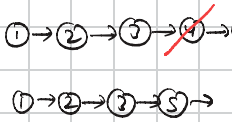
```
public ListNode addTwoNumbers(ListNode l1, ListNode l2) {

    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;

    int sum = 0, carry = 0;

    while(l1 != null || l2 != null){
        int p, q;

        if(l1 == null){
            p = 0;
        }else{
            p = l1.val;
        }

        if(l2 == null){
            q = 0;
        }else{
            q = l2.val;
        }

        sum = p + q + carry;
        carry = sum/10;

        curr.next = new ListNode(sum%10);
        curr = curr.next;

        if(l1 == null){
            l1 = null;
        }else{
            l1 = l1.next;
        }

        if(l2 == null){
            l2 = null;
        }else{
            l2 = l2.next;
        }

    }

    if(carry > 0){
        curr.next = new ListNode(1);
    }

    return dummy.next;
}
```

(43)

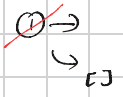
19. Remove Nth Node from End of List (Medium) (Linked List)

Given the head of a linked list, remove the n^{th} node from the end of the list and return its head.

Eg:- head = [1,2,3,4,5] n=2

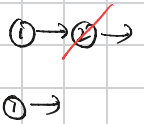


Eg2:- head = [1] n=1



[]

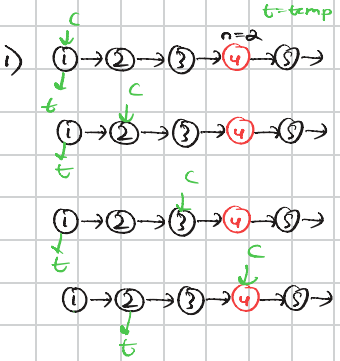
Eg3:- head = [1,2] n=1

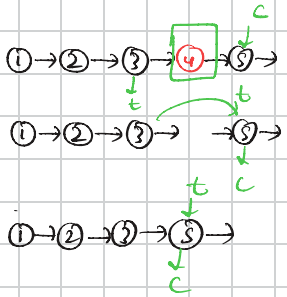


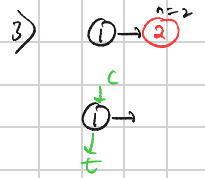
Code:-

```
public ListNode removeNthFromEnd(ListNode head, int n) {

    if(head.next == null) return null;

    ListNode curr = head;
    ListNode temp = head;

    int i = 1;
    while(i <= n){
        curr = curr.next;
        i++;
    }

    if(curr == null) return head.next;

    while(curr.next != null){
        curr = curr.next;
        temp = temp.next;
    }

    if(temp.next != null){
        temp.next = temp.next.next;
    }

    return head;
}
```

t = temp c = curr

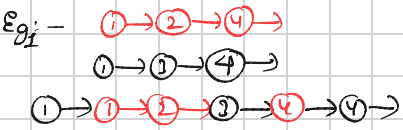
i) 

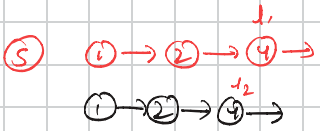




21. Merge Two Sorted Lists (Easy) (Linked List, Dummy Node)

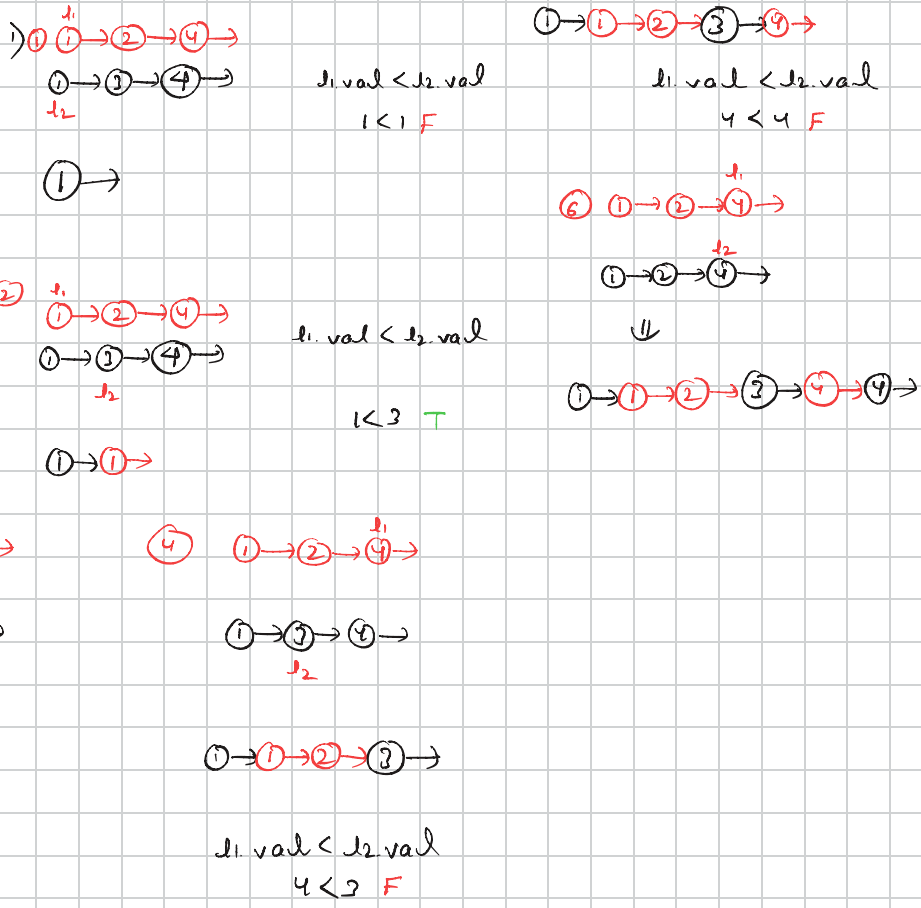
You are given the heads of two sorted linked lists list1 and list2. Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists. Return the head of the merged linked list.

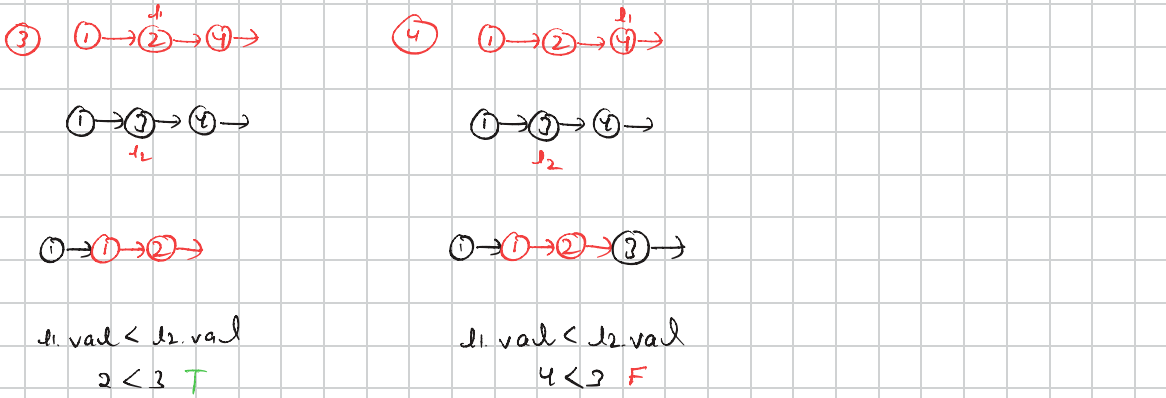




Code:-

```
public ListNode mergeTwoLists(ListNode
list1, ListNode list2) {
    ListNode dummy = null;

    if(list1 == null) return list2;
    if(list2 == null) return list1;

    if(list1.val < list2.val){
      dummy = list1;
      list1 = list1.next;
    }else{
      dummy = list2;
      list2 = list2.next;
    }

    ListNode curr = dummy;

    while(list1 != null && list2 != null){

      if(list1.val < list2.val){
        curr.next = list1;
        list1 = list1.next;
      }else{
        curr.next = list2;
        list2 = list2.next;
      }

      curr = curr.next;
    }

    if(list1 == null){
      curr.next = list2;
    }

    if(list2 == null){
      curr.next = list1;
    }

    return dummy;
  }
```



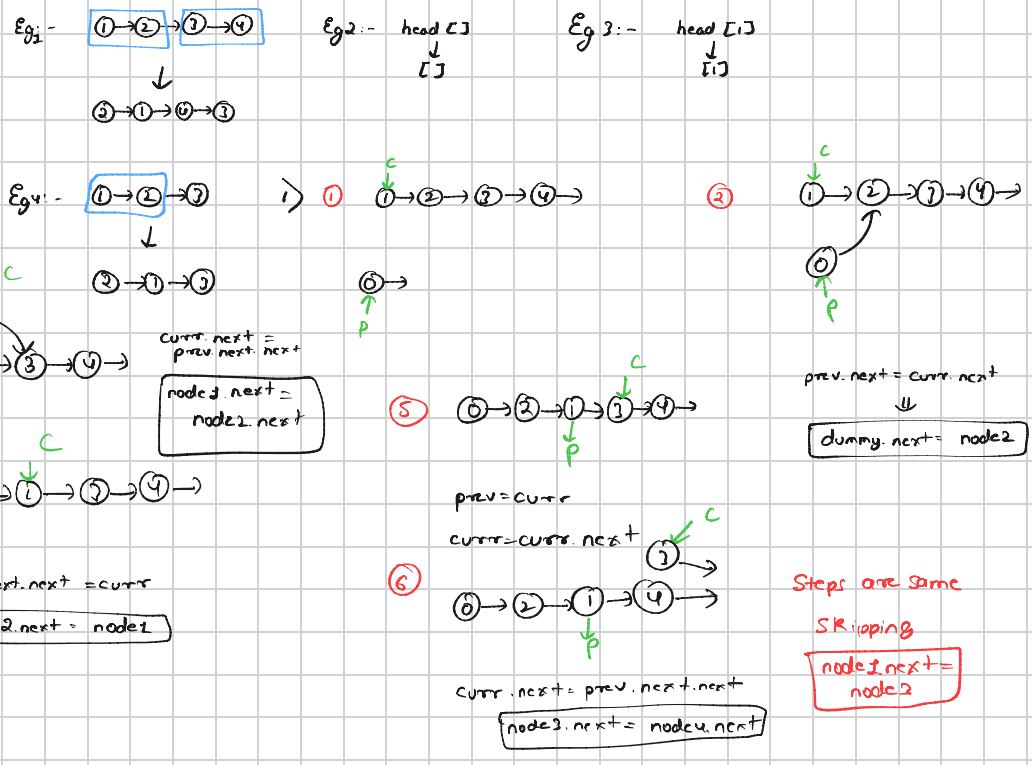


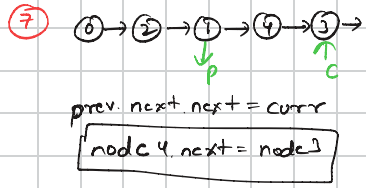
24. Swap Nodes in Pairs (Medium) (Linked List, Dummy Node, Swapping Algorithm)

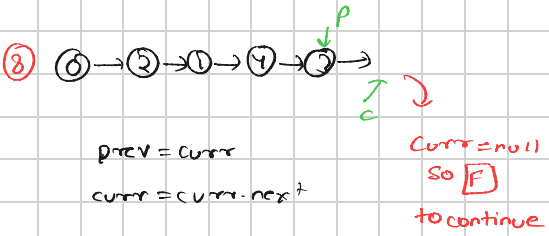
Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

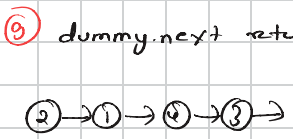
Code:-

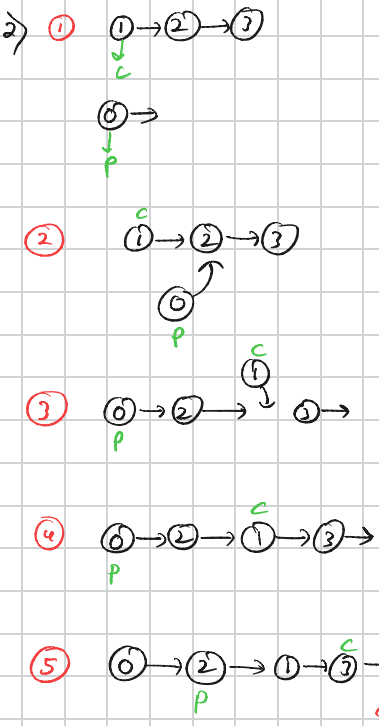
```
public static ListNode swapPairs(ListNode head) {
    if(head == null || head.next == null) return head;

    ListNode dummy = new ListNode(0);

    ListNode prev = dummy;
    ListNode curr = head;

    while(curr!= null && curr.next != null){
      prev.next = curr.next;
      curr.next = prev.next.next;
      prev.next.next = curr;
      prev = curr;
      curr = curr.next;
    }

    return dummy.next;
  }
```









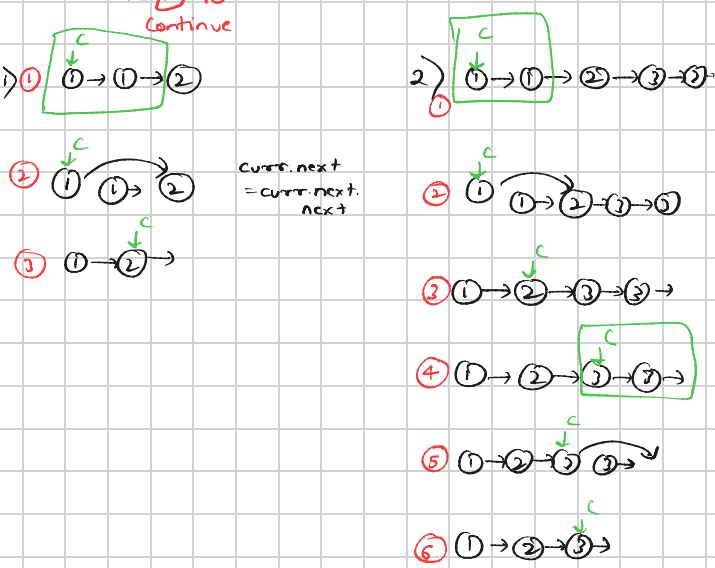


46

83. Remove Duplicates from Sorted List (Easy) (Linked List)

Eg 1:- 1→1→2 ⇒ 1→2

Eg 2:- 1→1→2→3→3 ⇒ 1→2→3



Code:-

```
public static ListNode deleteDuplicates(ListNode head) {
    if(head == null || head.next == null) return head;

    ListNode curr = head;

    while(curr != null){
        while(curr.next != null && curr.val == curr.next.val){
            curr.next = curr.next.next;
        }
        curr = curr.next;
    }

    return head;

}
```

47

206. Reverse Linked List (Easy) (Linked List, Swapping Method)

Given the head of a singly linked list, reverse the list, and return the reversed list.

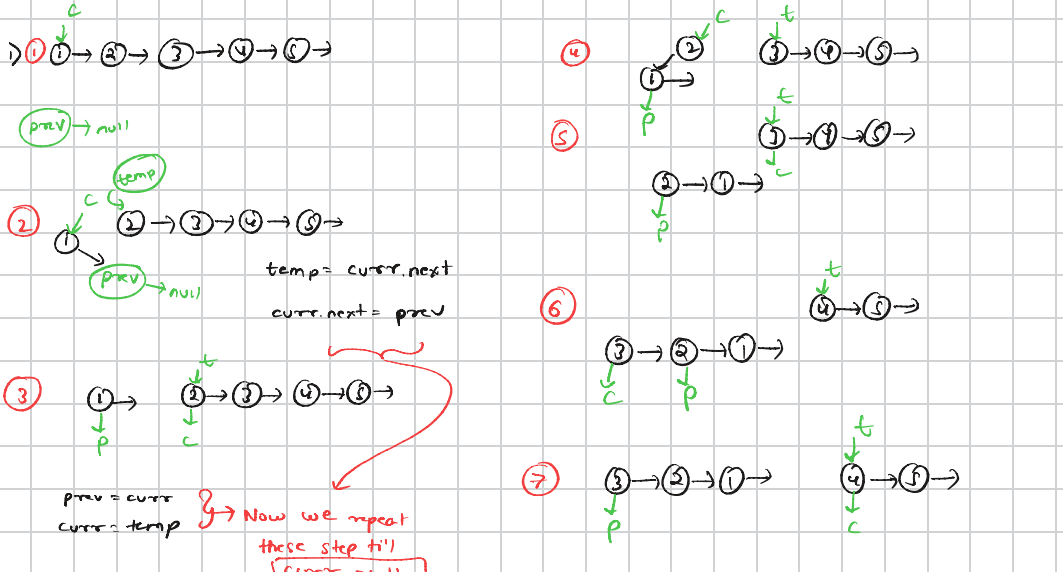
Code:-

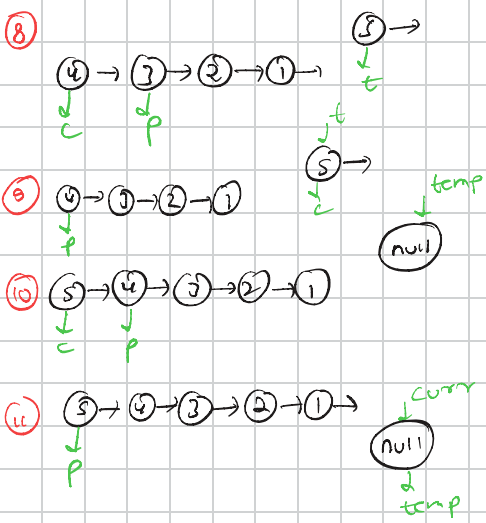
```
public ListNode reverseList(ListNode head) {
    if(head == null || head.next == null) return head;

    ListNode prev = null;
    ListNode curr = head;

    while(curr != null){
        ListNode temp = curr.next;
        curr.next = prev;
        prev = curr;
        curr = temp;
    }

    return prev;

}
```

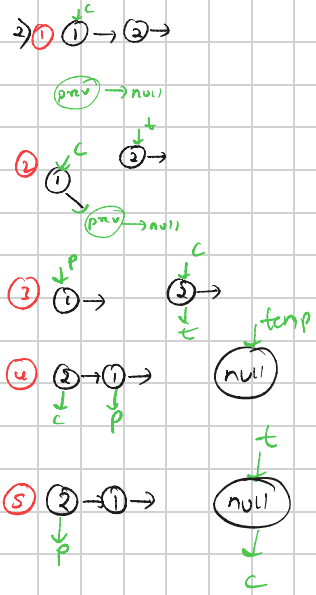
Eg1:- 1→2→3→4→5 ⇒ 5→4→3→2→1

Eg2:- 1→2 ⇒ 2→1

Eg3:- head [] ⇒ []







(98)

92. Reverse Linked List - II (Medium) (Linked List, Dummy Node, Swapping Algorithm)

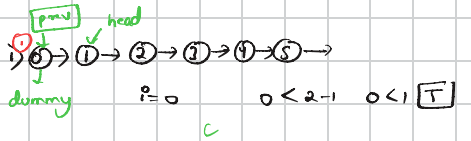
Given the head of a singly linked list and two integers left and right where left <= right, reverse the nodes of the list from position left to position right, and return the reversed list.

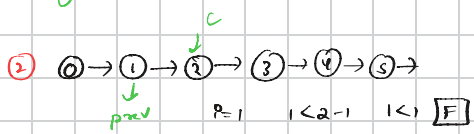
Code :-

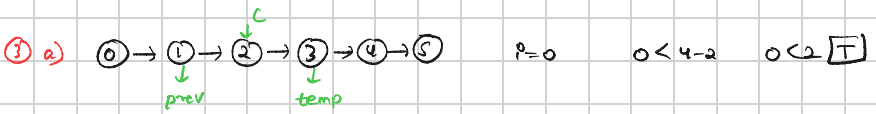
```
public static ListNode reverseBetween(ListNode head, int left,
int right) {
    if(head == null || left == right) return head;

    ListNode dummy = new ListNode(0);

    dummy.next = head;
    ListNode prev = dummy;

    for(int i = 0; i < left - 1; i++){
        prev = prev.next;
    }

    ListNode curr = prev.next;

    for(int i = 0; i < right - left; i++){
        ListNode temp = curr.next;
        curr.next = temp.next;
        temp.next = prev.next;
        prev.next = temp;
    }

    return dummy.next;
}
```

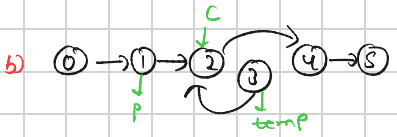
Eg1 :- 1→2→3→4→5→ ⇒ 1→4→3→2→5→

Eg2 :- head = [5] left = 1, right = 1 ⇒ [5]

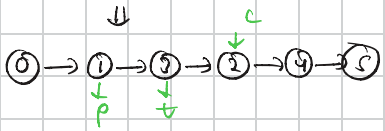


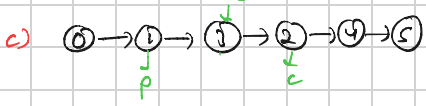


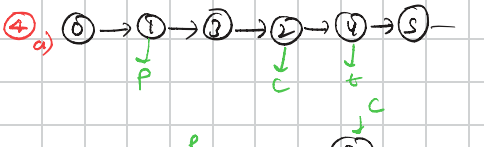




curr.next = temp.next
temp.next = prev.next







curr.next = temp.next
temp.next = prev.next

(5) Now p=2 2<4-2 2<2 F

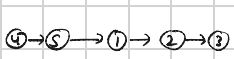
So we got our ans

we return dummy.next

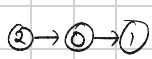
1→4→3→2→5→

61. Rotate List (Medium) (Linked List, Swapping Algorithm)

Eg1:- 1→2→3→4→5 K=2



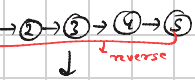
Eg2:- 0→1→2 K=4



Code:-

```
public ListNode reverseBetween(ListNode head, int left, int right) {
    if (head == null || left == right) {
        return head;
    }

    ListNode dummy = new ListNode(0);
    dummy.next = head;

    ListNode prev = dummy;

    for (int i = 1; i < left; i++) {
        prev = prev.next;
    }

    ListNode cur = prev.next;

    for (int i = 0; i < right - left; i++) {
        ListNode temp = cur.next;
        cur.next = temp.next;
        temp.next = prev.next;
        prev.next = temp;
    }

    return dummy.next;
}

public int getLength(ListNode head) {
    int count = 0;

    while (head != null) {
        count++;
        head = head.next;
    }

    return count;
}

public ListNode rotateRight(ListNode head, int k) {
    if (head == null || head.next == null) {
        return head;
    }

    int n = getLength(head);
    k %= n;

    if (k == 0) {
        return head;
    }

    head = reverseBetween(head, 1, n);
    head = reverseBetween(head, 1, k);
    head = reverseBetween(head, k + 1, n);

    return head;
}
```

1) 1→2→3→4→5 K=2 (reverse 1 to K)



K = k%n = 2%5 = 2

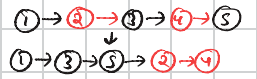
2) 0→1→2 K=4 (reverse 1 to K)

K = k%n = 4%3 = 1

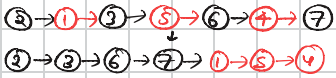
328. Odd Even Linked List (Medium) (Linked List, Two Pointers)

Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list. The first node is considered odd, and the second node is even, and so on. Note that the relative order inside both the even and odd groups should remain as it was in the input. You must solve the problem in O(1) extra space complexity and O(n) time complexity.

Eg1:- 1→2→3→4→5 → 1→3→5→2→4



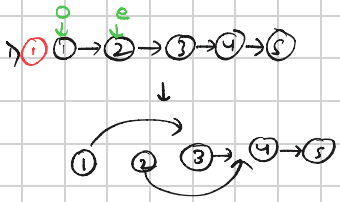
Eg2:- 2→1→3→5→6→4→7 → 2→3→6→7→1→5→4



Code:-

```
public ListNode oddEvenList(ListNode head) {

    if(head == null || head.next == null) return head;

    ListNode odd = head, even = head.next;
    ListNode evenHead = even;

    while(even != null && even.next != null){
        odd.next = even.next;
        odd = odd.next;
        even.next = odd.next;
        even = even.next;
    }

    odd.next = evenHead;
    return head;

}
```

1)



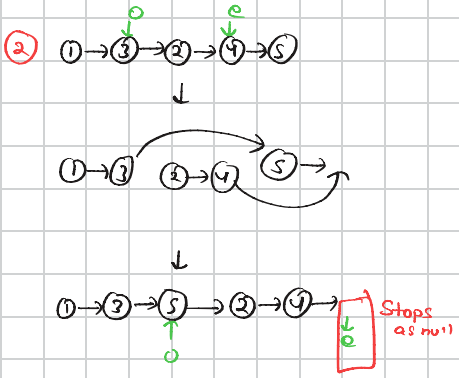
odd.next = even.next

odd = odd.next

even.next = odd.next

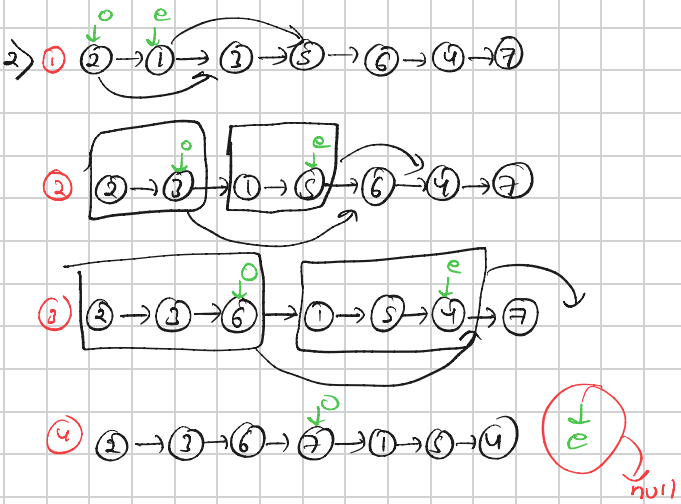
even = even.next

2)



Stops as null

2)



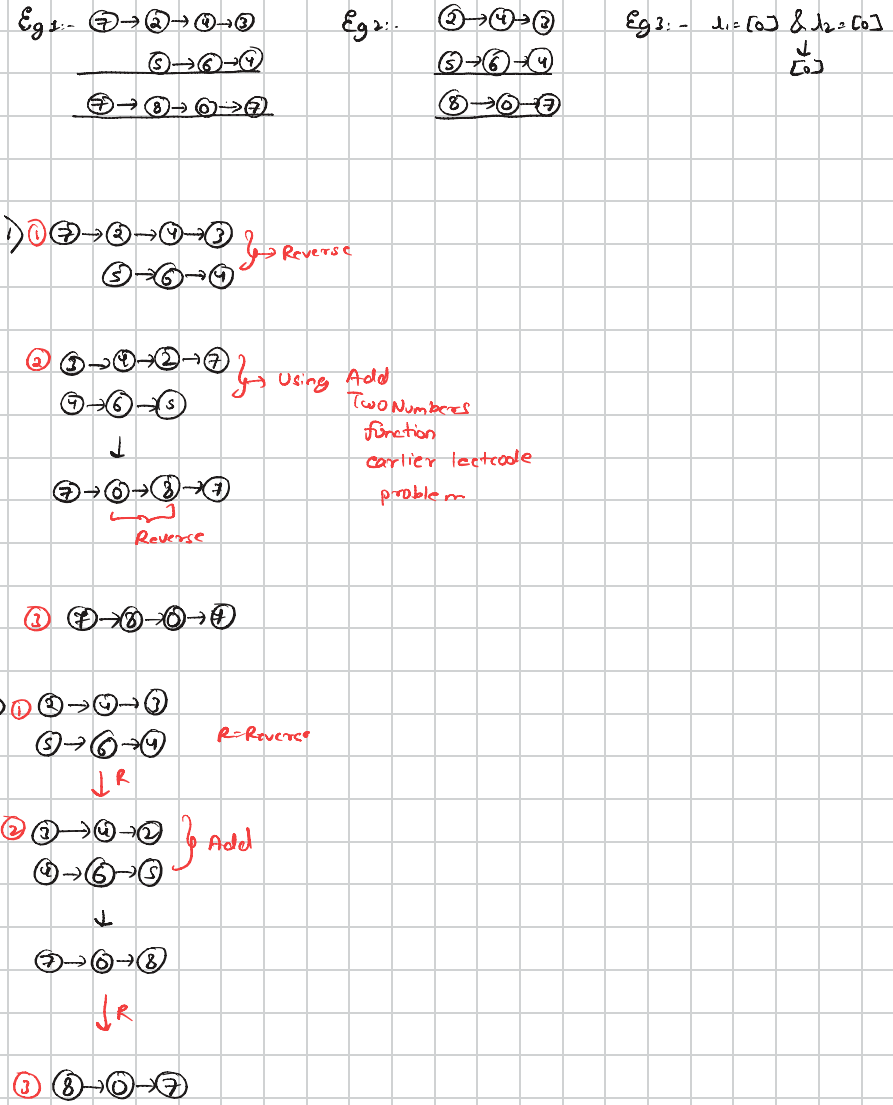
(51)

445. Add Two Numbers - II (Medium) (Linked List, Swapping Algorithm)

You are given two non-empty linked lists representing two non-negative integers. The most significant digit comes first and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list. You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Code:-

```
public static ListNode reverse(ListNode head) {

    ListNode prev = null;
    ListNode curr = head;

    while (curr != null) {

        ListNode temp = curr.next;

        curr.next = prev;

        prev = curr;
        curr = temp;
    }

    return prev;
}

public static ListNode addTwoNumbers(ListNode l1, ListNode l2) {

    // Reverse both linked lists
    l1 = reverse(l1);
    l2 = reverse(l2);

    ListNode dummyHead = new ListNode(0);
    ListNode curr = dummyHead;

    int carry = 0;

    while (l1 != null || l2 != null) {

        int p;

        if (l1 == null) {
            p = 0;
        } else {
            p = l1.val;
        }

        int q;

        if (l2 == null) {
            q = 0;
        } else {
            q = l2.val;
        }

        int sum = p + q + carry;

        carry = sum / 10;

        curr.next = new ListNode(sum % 10);
        curr = curr.next;

        if (l1 != null) {
            l1 = l1.next;
        }

        if (l2 != null) {
            l2 = l2.next;
        }
    }

    if (carry > 0) {
        curr.next = new ListNode(carry);
    }

    // Reverse the answer
    return reverse(dummyHead.next);
}
```



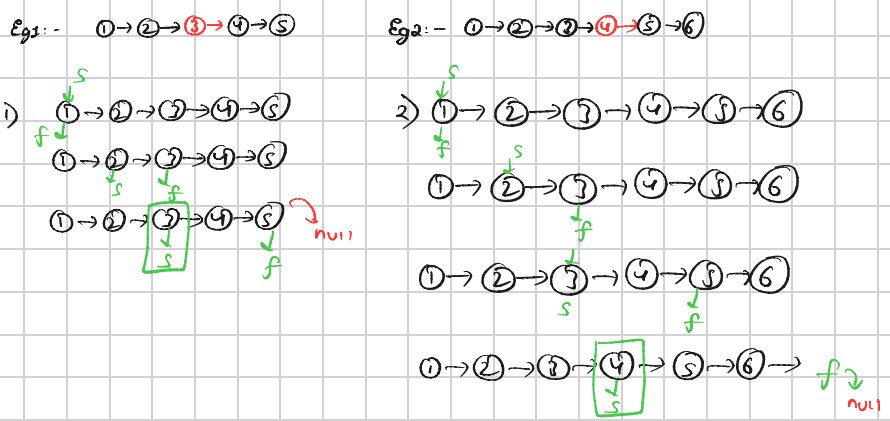
(52)

876. Middle of the Linked List (Easy) (Linked List, Two Pointers)

Given the head of a singly linked list, return the middle node of the linked list. If there are two middle nodes, return the second middle node.

Code:-

```
public ListNode middleNode(ListNode head) {
    if(head == null ) return null;

    ListNode slow = head;
    ListNode fast = head;

    while(fast != null && fast.next != null){
        slow = slow.next;
        fast = fast.next.next;
    }

    return slow;
}
```



2058. Find the Minimum and Maximum Number of Nodes Between Critical Points (Medium) (Linked List, Pointer Manipulation)

A critical point in a linked list is defined as either a local maxima or a local minima. A node is a local maxima if the current node has a value strictly greater than the previous node and the next node. A node is a local minima if the current node has a value strictly smaller than the previous node and the next node. Note that a node can only be a local maxima/minima if there exists both a previous node and a next node. Given a linked list head, return an array of length 2 containing [minDistance, maxDistance] where minDistance is the minimum distance between any two distinct critical points and maxDistance is the maximum distance between any two distinct critical points. If there are fewer than two critical points, return [-1, -1].

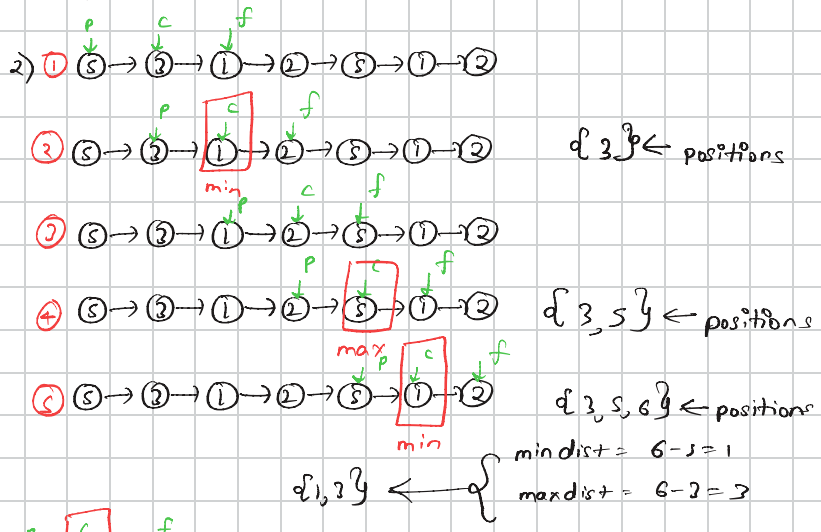
Code:-

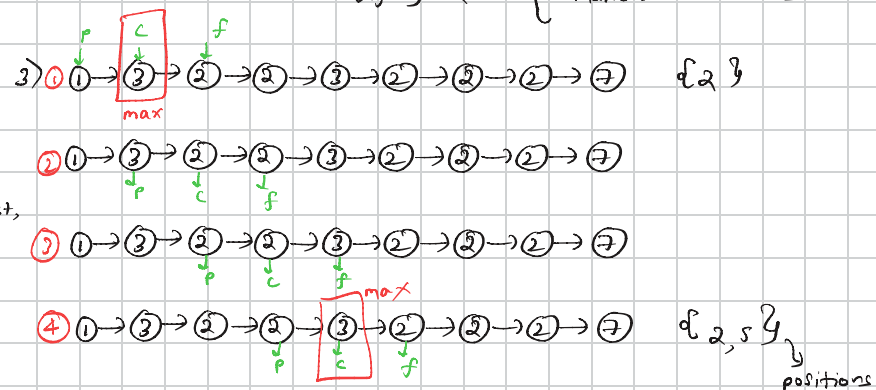
```
public int[] nodesBetweenCriticalPoints(ListNode head) {
    if(head == null || head.next == null || head.next.next == null) return new int[]{-1,-1};

    ArrayList<Integer> criticalPointsPositions = new ArrayList<>();

    ListNode prev = head;
    ListNode curr = head.next;
    ListNode future = head.next.next;

    int position = 1;

    while(future != null){

        if((curr.val < prev.val && curr.val < future.val) || (curr.val > prev.val && curr.val > future.val) ) {
            criticalPointsPositions.add(position);
        }

        position++;

        prev = curr;
        curr = future;
        future = future.next;
    }

    if(criticalPointsPositions.size() < 2){
        return new int[]{-1,-1};
    }

    int minDistance = Integer.MAX_VALUE;
    int maxDistance = 0;

    for(int i = 1; i < criticalPointsPositions.size() ; i++){
        int distance = criticalPointsPositions.get(i) - criticalPointsPositions.get(i-1);
        minDistance = Math.min(distance,minDistance);
    }

    maxDistance = criticalPointsPositions.get(criticalPointsPositions.size()-1) - criticalPointsPositions.get(0);
    // Sorted Array List

    return new int[]{minDistance,maxDistance};

}
```

Eg1:- 3→1 Output = [-1,-1]

Eg2:- 5→3→1→2→5→1→2

Eg3:- 1→3→2→3→3→2→3→2→7



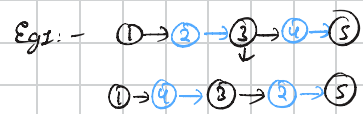


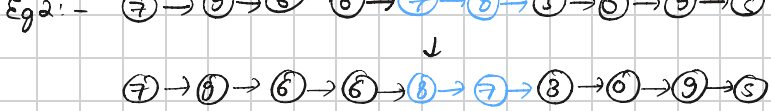
1721. Swapping Nodes in a Linked List (Medium) (Linked List, Two Pointers)

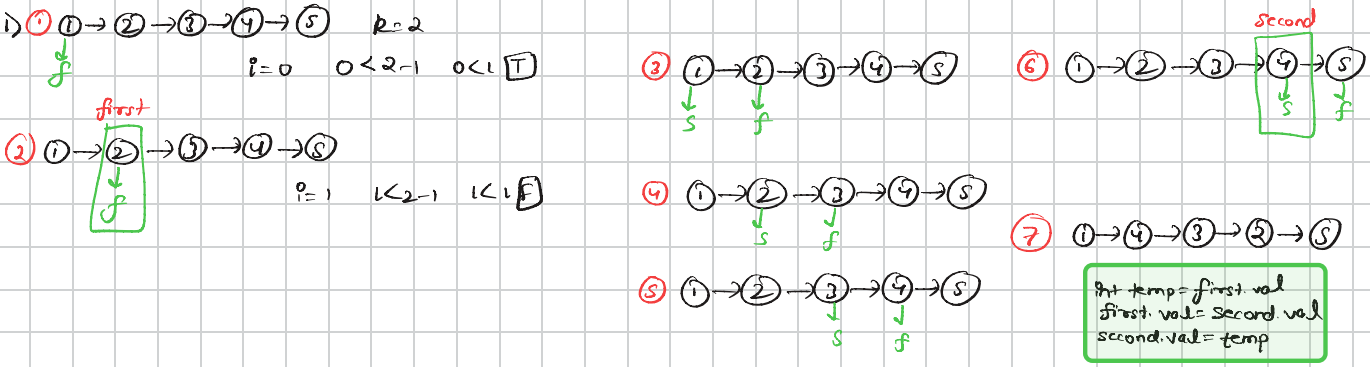
You are given the head of a linked list, and an integer k. Return the head of the linked list after swapping the values of the kth node from the beginning and the kth node from the end (the list is 1-indexed).

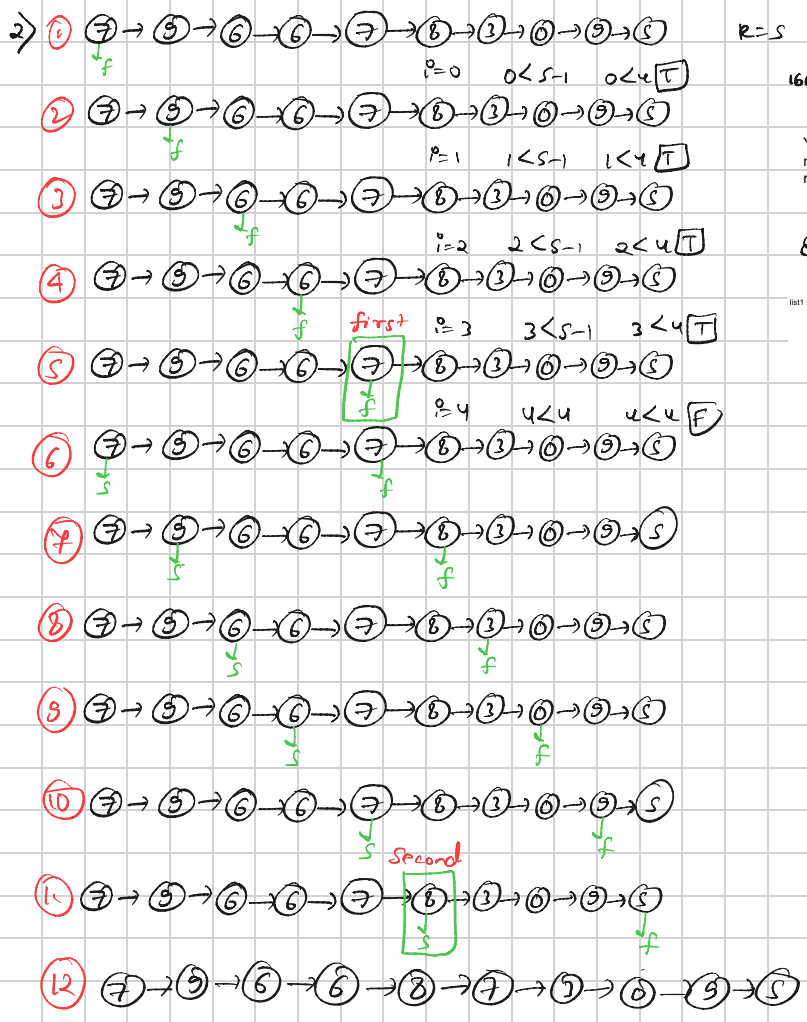
Code:-

```
public ListNode swapNodes(ListNode head, int k) {

    ListNode slow = head, fast = head, first = head , second = head;

    for(int i = 0; i < k - 1; i++){
        fast = fast.next;
    }

    first = fast;

    while(fast.next != null){
        fast = fast.next;
        slow = slow.next;
    }

    second = slow;

    int temp = first.val;
    first.val = second.val;
    second.val = temp;

    return head;
}
```









R=5

i=0 0<5-1 0<4 T

i=1 1<5-1 1<4 T

i=2 2<5-1 2<4 T

first

i=3 3<5-1 3<4 T

i=4 4<4 4<4 F

slow

fast

Second

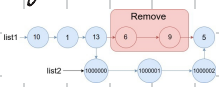
Removed

55

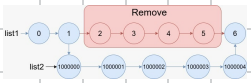
1669. Merge In Between Linked Lists (Medium) (Linked List, Two Pointers)

You are given two linked lists: list1 and list2 of sizes n and m respectively.Remove list1's nodes from the ath node to the bth node, and put list2 in their place. The blue edges and nodes in the following figure indicate the result:

Eg1:- a=3 b=4



Eg2:- a=2 b=5



Code:-

```
public ListNode mergeInBetween(ListNode list1, int a, int b, ListNode list2) {
    ListNode slow = list1;
    ListNode fast = list1;

    for(int i = 1; i <= b ; i++){
        if( i > a -1){
            fast = fast.next;
        }else{
            slow = slow.next;
            fast = fast.next;
        }
    }

    slow.next = list2;

    while(slow != null && slow.next != null){
        slow = slow.next;
    }

    slow.next = fast.next;
    return list1;
}
```

int temp = first.val
first.val = second.val
second.val = temp

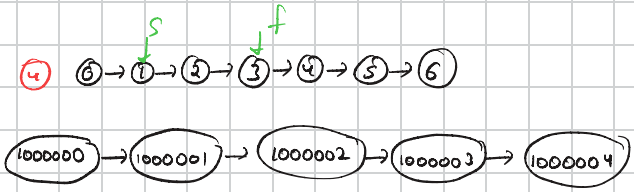
a=3 b=4

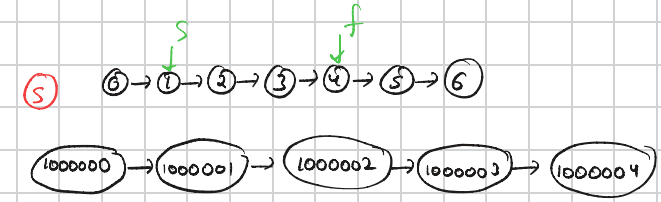
i=1 1<=4 T 1>3-1 1>2 F

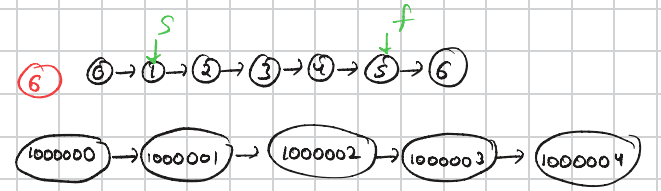
i=2 2<=4 T 2>2 F

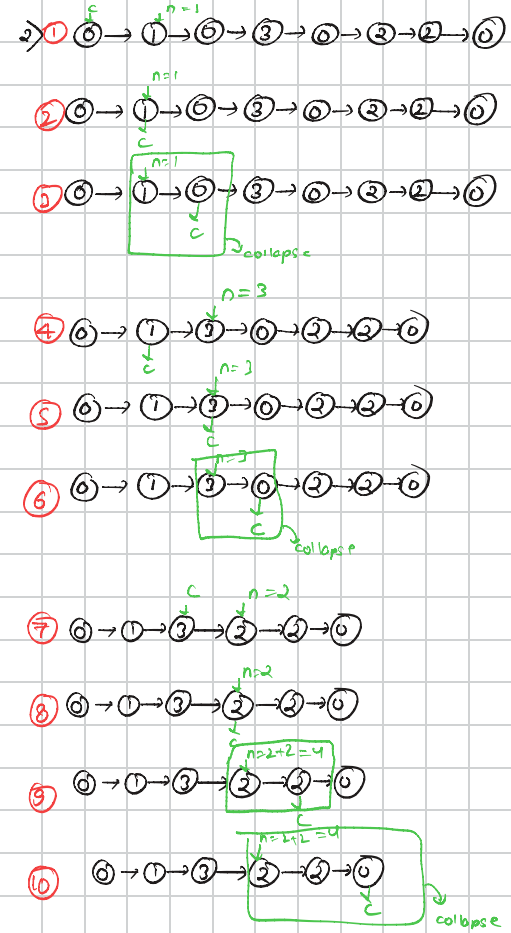
i=3 3<=4 T 3>2 T

i=4 4<=4 T 4>2 T









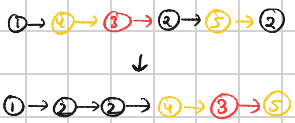
86. Partition List (Medium) (Linked List, Dummy Node)

Given the head of a linked list and a value x, partition it such that all nodes less than x come before nodes greater than or equal to x.
You should preserve the original relative order of the nodes in each of the two partitions.

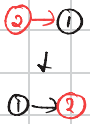
Code:-

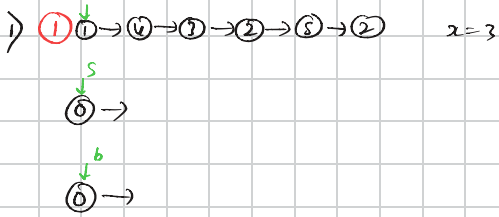
```
public ListNode partition(ListNode head, int x) {
    ListNode slist = new ListNode();
    ListNode blist = new ListNode();
    ListNode small = slist;
    ListNode big = blist;

    ListNode curr = head;

    while (curr != null) {
      if (curr.val < x) {
          small.next = curr;
          small = small.next;
      } else {
          big.next = curr;
          big = big.next;
      }

      curr = curr.next;
    }

    small.next = blist.next;
    big.next = null;

    return slist.next;
}
```

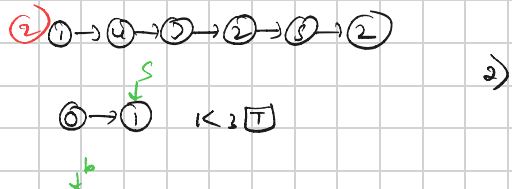
Eg1:-

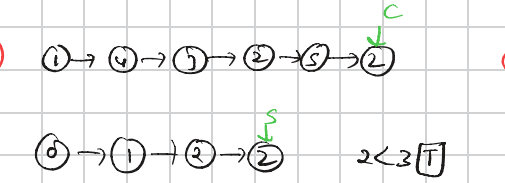


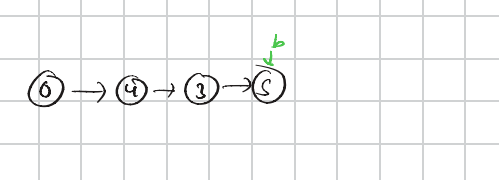
Eg2:-

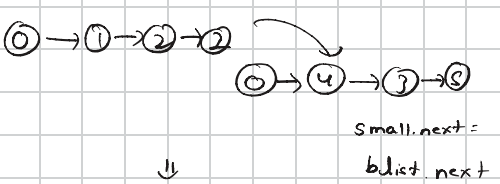


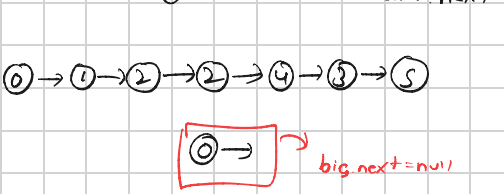












58

2130. Maximum Twin Sum of a Linked List (Medium) (Linked List, Pointer Manipulation)

In a linked list of size n, where n is even, the ith node (0-indexed) of the linked list is known as the twin of the (n-1-i)th node, if 0 <= i <= (n / 2) - 1. For example, if n = 4, then node 0 is the twin of node 3, and node 1 is the twin of node 2. These are the only nodes with twins for n = 4. The twin sum is defined as the sum of a node and its twin. Given the head of a linked list with even length, return the maximum twin sum of the linked list.

Code:-

```
public int pairSum(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;
    ListNode prev = null;

    while(fast != null && fast.next != null){
        fast = fast.next.next;

        ListNode temp = slow.next;
        slow.next = prev;
        prev = slow;
        slow = temp;
    }

    int sum = 0;

    while(slow != null){
        sum = Math.max(sum, prev.val + slow.val);
        prev = prev.next;
        slow = slow.next;

    }

    return sum;
}
```

Eg1:- 5 → 4 → 2 → 1 (0, 1, 2, 3)

output :- 6 (5+1) = 6, (4+2) = 6

Eg2:- 4 → 2 → 3 → 3 (0, 1, 2, 3)

output :- 7 (4+3) = 7, (2+2) = 4

Eg3 1 → 100000 (0, 1)

Output :- 100001

1) ① 5 → 4 → 2 → 1 (s, f)

② p → null; 5 (p); 4 → 2 → 1 (s, f)

③ 5 → 4 → 2 → 1; 5 → null (p); 4 (t); 2 → 1 (s); fast → null

④ 2 → 1 (s); 4 → 5 (p); Sum = (0, 2+4) = 6

⑤ 2 → 1 (p, s); 4 → 5; Sum = (6, 1+5) = 6

temp = slow.next

slow.next = prev

prev = slow

slow = temp

2) ① 4 → 2 → 2 → 3 (s, f)

② prev → null; 4 (p); 2 → 2 → 3 (t, f, s)

③ 4 ← 2; 2 → 3 (p, t, s); fast → null

④ 4 → 2 (p, s); Sum = (0, 2+2) = 4

⑤ 2 → 4 (p, s); 2 → 3; Sum = (4, 4+3) = 7

59

143. Reorder List (Medium) (Linked List, Two Pointers, Swapping Algorithm)

You are given the head of a singly linked-list. The list can be represented as:
L0 → L1 → … → Ln - 1 → Ln
Reorder the list to be on the following form:
L0 → Ln → L1 → Ln - 1 → L2 → Ln - 2 → …
You may not modify the values in the list's nodes. Only nodes themselves may be changed.

Code:-

```
public void reorderList(ListNode head) {
    if (head == null || head.next == null) {
        return;
    }

    ListNode middle = findMiddle(head);
    ListNode secondHalf = splitAfter(middle);
    secondHalf = reverseList(secondHalf);

    mergeHalves(head, secondHalf);
}

private ListNode findMiddle(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    while (fast.next != null && fast.next.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    return slow;
}

private ListNode splitAfter(ListNode middle) {
    ListNode secondHalf = middle.next;
    middle.next = null;
    return secondHalf;
}

private ListNode reverseList(ListNode head) {
    ListNode previous = null;
    ListNode current = head;

    while (current != null) {
        ListNode next = current.next;
        current.next = previous;
        previous = current;
        current = next;
    }

    return previous;
}

private void mergeHalves(ListNode first, ListNode second) {
    while (second != null) {
        ListNode nextFirst = first.next;
        ListNode nextSecond = second.next;

        first.next = second;
        second.next = nextFirst;

        first = nextFirst;
        second = nextSecond;
    }
}
```

Eg1:- 1 → 2 → 3 → 4 ⇒ 1 → 4 → 2 → 3

Eg2:- 1 → 2 → 3 → 4 → 5 ⇒ 1 → 5 → 2 → 4 → 3

1) ① Finding the middle

a) 1 → 2 → 3 → 4 (s); 1 (p)

b) 1 → 2 → 3 → 4 (s, f); null

slow = 2

② Splitting

Second half ⇒ 3 → 4 →

and we break the linked

1 → 2 →

③ Reverse second half

4 → 3 →

④ Merging

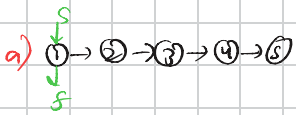
a) 1 → 2 → (s, f); 4 → 3 → (s, f)

b) 1 → 2 → (f); 4 (s); 3 (s)

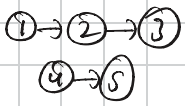
c) 1 → 4 → 2; 3; null

1 → 4 → 2 → 3

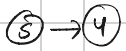
2) ① Finding the middle

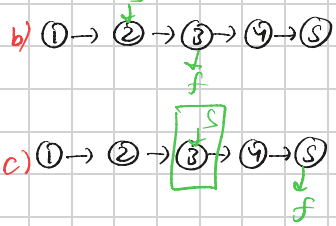


② Splitting

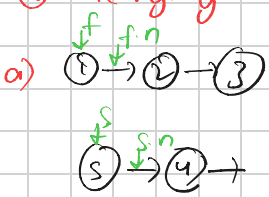


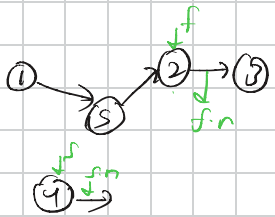
③ Reverse of second half

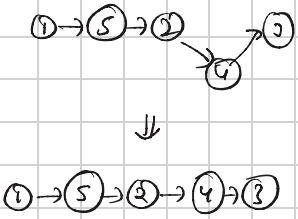




④ Merging







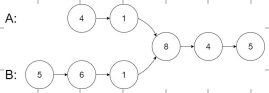
Slow = 3

60

160. Intersection of Two Linked List (Easy) (Linked Lists)

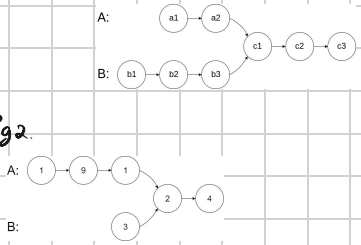
Given the heads of two singly linked-lists headA and headB, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return null.

Eg 1.



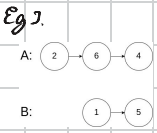
Input: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,6,1,8,4,5], skipA = 2, skipB = 3
Output: Intersected at '8'
Explanation: The intersected node's value is 8 (note that this must not be 0 if the two lists intersect).
From the head of A, it reads as [4,1,8,4,5]. From the head of B, it reads as [5,6,1,8,4,5]. There are 2 nodes before the intersected node in A; There are 3 nodes before the intersected node in B.
- Note that the intersected node's value is not 1 because the nodes with value 1 in A and B (2nd node in A and 3rd node in B) are different node references. In other words, they point to two different locations in memory, while the nodes with value 8 in A and B (3rd node in A and 4th node in B) point to the same location in memory.

Eg2.



Input: intersectVal = 2, listA = [1,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1
Output: Intersected at '2'
Explanation: The intersected node's value is 2 (note that this must not be 0 if the two lists intersect).
From the head of A, it reads as [1,9,1,2,4].
From the head of B, it reads as [3,2,4]. There are 3 nodes before the intersected node in A; There are 1 node before the intersected node in B.

Eg 3.

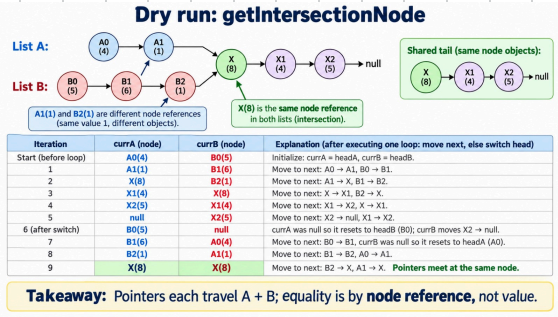


Input: intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2
Output: No intersection
Explanation: From the head of A, it reads as [2,6,4]. From the head of B, it reads as [1,5]. Since the two lists do not intersect, intersectVal must be 0, while skipA and skipB can be arbitrary values.
Explanation: The two lists do not intersect, so return null.

Code: -

```
public ListNode getIntersectionNode(ListNode
headA, ListNode headB) {
    ListNode currA = headA;
    ListNode currB = headB;

    while(currA != currB){
        if(currA != null){
            currA = currA.next;
        }else{
            currA = headB;
        }

        if(currB != null){
            currB = currB.next;
        }else{
            currB = headA;
        }
    }

    return currA;
}
```

1)



2)